

Algorithm for file updates in Python

Project description

In this project, I'm working as a security specialist at a health care company. My job is to keep the allow list of IP addresses up to date. This list controls who can access a restricted network that contains patient information. Some employees no longer need access, and their IP addresses are listed in a separate remove list. The goal is to write a Python script that compares both lists, finds any matching IP addresses, and removes them from the allow list file. This helps ensure that only authorized employees can reach the restricted system.

Open the file that contains the allow list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

```
import_file = "allow_list.txt"
```

This line assigns the file name as a string to the variable `import_file`.

```
with open(import_file, "r") as file:
```

This is a `with statement`, which is used to open and work with files safely.

`open()` is a built-in Python function that opens a file.

"r" is the mode for reading the file.

`file` is the variable that stores the opened file object.

The `with statement` ensures that the file is properly closed after the block is executed, even if an error occurs. This setup is the first step in building an algorithm that reads the allow list and prepares it for updates.

Read the file contents

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

To read the contents of a file in Python, we use the `open()` function together with the `.read()` method inside a `with` statement.

`open("filename", "r")` — opens the file in **read mode** ("r"). `with` — ensures the file is properly opened and automatically closed after you're done.

`.read()` — reads the entire content of the file as one string.

The result is stored in a variable (e.g., `ip_addresses`) so you can use it later in your code.

Convert the string into a list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Display `ip_addresses`
```

When reading a file using Python, it stores the entire content as one long string. To work with individual IP addresses, it needs to split that string into a list.

To do this, you use the `.split()` method.

Iterate through the remove list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Display `element` in every iteration
    print(element)
```

To iterate through the `remove_list`, I use a `for` loop in Python. This allows you to go through each IP address in the list one by one and perform actions with it — for example, checking if it appears in the allow list.

Remove IP addresses that are on the remove list

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`
for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,
    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)
```

In this step, the goal is to check each IP address from the `allow_list` and remove it if it appears in the `remove_list`. The code in my screenshot does this by using a **for-loop** combined with an **if-statement**:

- The loop goes through each element in `ip_addresses`.
- The `if` condition checks whether the current element is also in `remove_list`.
- If the condition is true, the `.remove()` method deletes that IP from `ip_addresses`.

This logic ensures that any IP address found in both lists is removed from the allow list.

Update the file with the revised list of IP addresses

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`
        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = " ".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w"):

    # Rewrite the file, replacing its contents with `ip_addresses`
    file.write(ip_addresses)
```

To **update the file with the revised list of IP addresses**, the script performs the following final step:

- It opens the original file (`allow_list.txt`) in **write mode** using `with open(import_file, "w") as file:`. This mode clears the existing contents of the file.
- Then it writes the **filtered list of IP addresses**, which has been converted back into a string using `" ".join(ip_addresses)`, into the file using `file.write(ip_addresses)`.

This ensures that the file now contains only the IP addresses that are still allowed access—those not present in the `remove_list`.

Summary

In this project, I walked through the full process of updating an allow list of IP addresses using Python. The script reads the file, turns its contents into a list, and compares it against a separate list of IPs that should no longer have access. Any matches are removed so the allow list stays accurate and secure. After filtering, the updated IPs are joined back into a string and written to the same file, replacing the old data. Overall, it's a straightforward algorithm that helps keep the restricted network protected and ensures that only the right people maintain access.